

POO

Pilares

Abstração

- é o processo de identificar e definir características essenciais de objetos do mundo real, "traduzindo" (representando) essas características em forma de classes e objetos.

Encapsulamento

- Encapsulamento é esconder os detalhes internos de uma classe e expor apenas o que faz sentido.
 - Exemplo: uma classe `ContaBancaria` não permite alterar o saldo diretamente; o saldo é privado e só pode mudar via métodos como depositar e sacar, que aplicam regras.
- Os principais objetivos do encapsulamento são:
 - Controle de Acesso: Controlar quem pode acessar e modificar os atributos da classe. Isso ajuda a evitar alterações indesejadas e a garantir que os dados sejam usados corretamente.
 - Proteção dos Dados: Garantir que os dados internos da classe não sejam corrompidos ou acessados indevidamente.
 - Flexibilidade: Permitir que a implementação interna da classe seja alterada sem afetar o código que a utiliza. Isso é possível porque a interface pública (métodos públicos) permanece a mesma.

Herança

- é o princípio que permite que uma classe (subclasse) herde os atributos e métodos de outra classe (superclasse), facilitando a reutilização de código, a organização hierárquica e a promoção de relações entre objetos.
- Quando você possui duas ou mais classes com atributos e métodos em comum, a herança facilita a criação de uma estrutura hierárquica, onde uma classe "pai" define os elementos comuns que serão herdados pelas classes "filhas".
- Em Java utilizamos a palavra-chave **extends** para indicar que uma Classe herda as informações de outra.

Polimorfismo

- é o princípio a partir do qual as classes derivadas de uma única classe base são capazes de invocar os métodos que, embora apresentem a mesma assinatura, comportam-se de maneira diferente para cada uma das classes derivadas.

Polimorfismo Estático

- O polimorfismo estático, também conhecido como sobrecarga de método ou polimorfismo de compile-time, é um conceito de POO em que uma classe pode ter vários métodos com o mesmo nome, mas com diferentes parâmetros.

Polimorfismo Dinâmico

- O polimorfismo dinâmico, também conhecido como polimorfismo de tempo de execução ou sobrescrita de método, permite que objetos de diferentes classes, que compartilham uma relação de herança, respondam de maneira específica a chamadas de métodos comuns.
- A sobrescrita de método é uma forma de implementar o polimorfismo dinâmico, na qual uma classe filha (subclasse) redefine um método previamente definido na classe pai (superclasse).
- A subclasse fornece sua própria implementação do método, alterando seu comportamento sem mudar a assinatura do método (nome, parâmetros e tipo de retorno).
- Com o polimorfismo, é possível reutilizar e adaptar comportamentos em diferentes contextos, permitindo que classes e métodos sejam usados de maneira flexível, sem perder a consistência da interface ou lógica comum.

Classes

- Classes basicamente são moldes para criar objetos, podendo conter variáveis e métodos. Os objetos criados a partir de uma classe, podem conter seus respectivos valores para as variáveis e podem executar os métodos.

Veja o exemplo abaixo:

```
public class Carro {
    public String name;
    public String model;
    public int year;
}

// Em outro arquivo:
package academy.devdojo.maratonajava.javacore.Aintroducaoclasses.test;

import academy.devdojo.maratonajava.javacore.Aintroducaoclasses.dominio.Carro;

public class CarrosTest {
    public static void main(String[] args) {
        Carro golBolinha = new Carro();
        Carro unoMille = new Carro();

        golBolinha.model = "Volkswagen Gol G2";
        golBolinha.year = 1995;
        golBolinha.name = "Gol Bolinha";

        unoMille.model = "Fiat Uno Mille";
        unoMille.name = "Uno Mille";
        unoMille.year = 1990;

        System.out.println("Nome: " + golBolinha.model + " Ano: " +
golBolinha.year + " Modelo: " + golBolinha.model);
        // Imprime: Nome: Volkswagen Gol G2 Ano: 1995 Modelo: Volkswagen Gol G2
    }
}
```

```

        System.out.println("-----");
        System.out.println("Nome: " + unoMille.model + " Ano: " + unoMille.year +
" Modelo: " + unoMille.model);
        // Imprime: Nome: Fiat Uno Mille Ano: 1990 Modelo: Fiat Uno Mille
    }
}

```

No exemplo acima, criamos duas classes: **Carro** e **CarrosTest**. A primeira serve para criar objetos e armazenar seus respectivos tipos de variáveis, enquanto a segunda define valores para as variáveis baseado na classe **Carro** e imprime seus respectivos valores.

Métodos

- Metodos basicamente representam uma ação que o objeto pode executar. Os métodos podem ou não receber parametros e também podem ou não retornar um valor. No exemplo a seguir:

```

package academy.devdojo.maratonajava.javacore.Bintroducaometodos.dominio;

public class Calculadora {

    public void somaDoisNumeros(int numberA, int numberB){
        int soma = numberA + numberB;
        System.out.println(soma);
    }

    public void subtraiDoisNumeros(int numberA, int numberB){
        int soma = numberA - numberB;
        System.out.println(soma);
    }

    public double divideDoisNumeros(double num1, double num2){
        return num1 / num2;
    }

    // No método subtraiDoisNumeros, o metodo é do tipo void e portanto não
    // retorna um valor, apenas executa uma ação. Já no metodo divideDoisNumeros, ele não
    // imprime o resultado, apenas calcula e devolve o valor usando a palavra-chave
    // return. Quem chama o método pode armazenar esse valor em uma variável, usar em
    // cálculos ou exibir na tela.
}

// Em outro arquivo:
package academy.devdojo.maratonajava.javacore.Bintroducaometodos.test;

import
academy.devdojo.maratonajava.javacore.Bintroducaometodos.dominio.Calculadora;

public class CalculadoraTest01 {
    public static void main(String[] args) {
        Calculadora calculadora = new Calculadora();
        calculadora.somaDoisNumeros(10, 20);
        // Retorna 30
    }
}

```

```

        calculadora.subtraiDoisNumeros(35, 8);
        // Retorna 27
    }
}

package academy.devdojo.maratonajava.javacore.Bintroducaometodos.test;

import
academy.devdojo.maratonajava.javacore.Bintroducaometodos.dominio.Calculadora;

public class CalculadoraTest02 {
    public static void main(String[] args) {
        Calculadora calculadora = new Calculadora();
        double result = calculadora.divideDoisNumeros(20, 2);
        System.out.println(result);
        // Retorna 10
    }
}

```

- Var Args
 - Ao percorrermos um array, existe um limite de elementos. Um recurso introduzido mais recentemente é o varargs. Ele permite que um metodo aceite um numero indefinido de argumentos, sendo que eles serao armazenados em um array. Porém ele tem algumas peculiaridades:

```

public void somaVarArgs(int... numeros){
    int soma = 0;
    for (int num : numeros){
        soma += num;
    }
    System.out.println(soma);
}

public static void main(String[] args) {
    Calculadora calculadora = new Calculadora();
    calculadora.somaVarArgs(1, 2, 3, 4, 5);
    // Retorna 15
}

```

- Com o Varargs, não é necessario criar uma nova variável com o array. Basta passar o array como argumento para o metodo.
- Ao contrário da outra forma, não podemos passar os ... após o tipo da variavel, por exemplo `int numeros...`, pois gera um erro de sintaxe.
- Também não podemos passar outros argumentos após esse array, por exemplo `int... numeros, int numberB`. Caso tivéssemos, é necessário que os argumentos fossem declarados depois do array, por exemplo `int numberB, int... numeros`.
- Caso associarmos um atributo antes do varArgs, por exemplo `int numberA, int... numeros`, o primeiro argumento será armazenado na variavel numberA e o restante será armazenado no array

numeros.

Vantagens de usar atributos privados ou protegidos (private e protected)

- Podemos garantir que o atributo seja acessado apenas pela classe em que ele foi criado
- Podemos garantir que o atributo seja acessado apenas pela classe em que ele foi criado ou por classes filhas
- Podemos garantir que o atributo possa ser apenas lido pelas classes filhas (por exemplo, ao calcular uma média, podemos apenas exibir o resultado, mas não poderemos alterar o valor.)

Sobrecarga de métodos

- Métodos com nomes iguais, mas com tipos ou quantidade de parâmetros diferentes. A sobrecarga de métodos é uma das maneiras pelas quais Java implementa o polimorfismo.

Construtores

- O construtor é o método da classe que permite que você inicie o objeto criado com valores iniciais. Por exemplo:

```
public Anime(String tipo, int episodios, String nome, String genero) {  
    this.tipo = tipo;  
    this.episodios = episodios;  
    this.nome = nome;  
    this.genero = genero;  
}  
  
// Novo objeto com valores iniciais  
Anime anime = new Anime("Mangá", 23, "Haikyuu", "Terror");
```

- O construtor deve ser declarado com o mesmo nome da classe e com o mesmo tipo de retorno.
- Normalmente, usamos um construtor para fornecer valores iniciais para as variáveis de instância definidas pela classe ou para executar algum outro procedimento de inicialização necessário à criação de um objeto totalmente formado.

Quando criamos um objeto:

1. Alocado espaço em memória pro objeto;
 2. Cada atributo de classe é criado e inicializado com o valor default ou o que for passado;
 3. Bloco de inicialização é executado
 4. Construtor é executado
- O bloco de inicialização é executado independentemente do construtor que for chamado

Classes abstratas

- Uma classe abstrata pode ser herdada, mas não pode ser instanciada.

- Métodos abstratos não possuem implementação, e devem ser obrigatoriamente implementados (@override) nas subclasses.
- Métodos abstratos são uma forma de garantir que todas as subclasses tenham um determinado comportamento definido

Palavra-chave final

- Para evitar que alguém da equipe sobrescreva o método e possa, "sem querer ou não", comprometer o cálculo, podemos utilizar a palavra chave "final".
- Quando queremos que uma classe não seja instanciada e sirva apenas para ser herdada utilizamos a palavra chave "abstract".
- Com a palavra chave "final" temos o contrário, podemos definir que uma classe não possa ser herdada.

Tipos genéricos

- Tipos genéricos em POO (Programação Orientada a Objetos) é um recurso que permite escrever classes, interfaces e métodos que são independentes do tipo de dado específico que será usado como parâmetro.
- Em outras palavras, em vez de escrever classes e métodos para lidar com tipos de dados específicos (como uma classe para lidar com inteiros, outra para lidar com strings, etc.), os tipos genéricos permitem escrever classes e métodos que podem funcionar com qualquer tipo de dados que seja especificado no momento da utilização.

Diamond Syntax

- A sintaxe diamond, também conhecida como operador diamante ou diamante vazio, é um recurso adicionado ao Java 7 que permite que o compilador infira o tipo de um objeto em uma declaração de um objeto genérico.
- Antes da introdução da sintaxe diamond, a declaração de um objeto genérico exigia que o tipo de objeto fosse especificado duas vezes - uma vez na declaração da variável e novamente na criação do objeto. Por exemplo, para criar um ArrayList que armazena objetos do tipo String, era necessário declará-lo assim:

- `Cesta<Fruta> cestaFrutas = new Cesta<Fruta>();`

- Com a sintaxe diamond, no entanto, é possível omitir a especificação do tipo de objeto na criação do objeto, deixando que o compilador infira o tipo com base no tipo especificado na declaração da variável. O exemplo acima poderia ser reescrito assim:

- `Cesta<Fruta> cestaFrutas = new Cesta<>();`

Interfaces

- Interfaces são um conceito da programação orientada a objetos que tem a ver com o comportamento esperado para uma ou um conjunto de classes.

- Interfaces definem o que uma classe deve fazer e não como. Assim, interfaces nem sempre possuem a implementação de métodos pois podem apenas declarar um conjunto de métodos, ou seja, o comportamento que uma ou um conjunto de classes deve ter.

Coleções em Java – API Collections

- Collections em Java se referem a um conjunto de classes e interfaces que fornecem uma estrutura para armazenar e manipular dados em grupo. As collections são parte do framework de coleções de Java e foram projetadas para serem flexíveis, eficientes e seguras para threads. O framework de coleções em Java inclui as seguintes interfaces principais:
 - Iterable: é a interface raiz de maioria das interfaces de coleções em Java.
 - Collection: Define métodos básicos para adicionar, remover e recuperar elementos de uma coleção.
 - List: uma coleção ordenada que permite elementos duplicados.
 - Set: uma coleção não ordenada que não permite elementos duplicados.
 - Map: uma coleção que armazena pares de chave-valor únicos. Cada elemento é acessado por meio de sua chave exclusiva.
- Além das interfaces acima, o framework de coleções em Java também inclui várias classes úteis para trabalhar com coleções, como ArrayList, LinkedList, HashSet, TreeSet e HashMap. Essas classes fornecem implementações concretas das interfaces acima e adicionam recursos adicionais, como iteração, ordenação e pesquisa.